ShipShape Codebase Orientation Checklist

Repository: US-Department-of-the-Treasury/ship **Completed:** 2026-03-09 **Goal:** Build a mental model of the entire system before measuring anything.

Phase 1: First Contact

1. Repository Overview

Clone the repo and get it running locally. Document every step, including anything not in the README.

The app was brought up using `pnpm docker:up` (not `docker-compose up -d` as the README states). Full findings and the improved README are appended to this document (see **Appendix A: Installation Process Evaluation** and **Appendix B: Improved README**). Key deviations from the README:

- Port 5432 was already allocated by another Docker container; `docker-compose.local.yml` (port 5433) was required.
- Seed and migrate run automatically inside the `Dockerfile.dev` CMD; no separate manual steps needed.
- The README has seed and migrate in the wrong order (seed before migrate).

Read every file in the docs/ folder. Summarize the key architectural decisions in your own words.

Document	Key Decision
`unified-document-model.md`	Everything (wiki, issue, project, sprint, person, weekly_plan, weekly_retro, standup, weekly_review) is a row in the `documents` table distinguished by `document_type`. This is the central architectural choice — Notion-style flat storage vs. separate tables per type.
`application-architecture.md`	Deliberate "boring technology" stack: Express, React, PostgreSQL, no ORM. Real-time via Yjs CRDTs over WebSocket. Monorepo with pnpm workspaces.
`document-model-conventions.md`	Terminology rules: "week" not "sprint", "Untitled" (never "Untitled Issue"), 4-panel editor layout canonical for all document types.
`sprint-documentation-philosophy.md` / `week-documentation-philosophy.md`	Plan-driven workflow: write plan before the week, retro after. Missing docs escalate visually from yellow to red. Accountability is visible, not blocking.
`ship-philosophy.md`	Three principles: Everything is a document, Server is truth, Boring technology. Explicitly rejects frameworks that would conflict with these.
`accountability-philosophy.md`	RACI model on programs/projects. Approval workflows on plans and retros. Compliance-grade audit logs.
`developer-workflow-guide.md`	Worktree-based multi-session development; `scripts/dev.sh` auto-creates DB and picks available ports.

Read the shared/ package. What types are defined? How are they used across frontend and backend?

The `shared/` package (`@ship/shared`) exports:

- **`document.ts`** — Core document model: `DocumentType` enum, `Document` base interface, typed variants (`IssueDocument` , `WikiDocument` , `ProjectDocument` , etc.), all `*Properties` interfaces (e.g., `IssueProperties` , `WeekProperties` , `PersonProperties`), `DocumentVisibility` , `BelongsTo` , `ApprovalTracking` , `computeICEScore()` .
- **`user.ts`** — User and workspace membership types.
- **`api.ts`** — API response envelope types (success/error wrappers).
- **`auth.ts`** — Auth-related types.
- **`workspace.ts`** — Workspace types.
- **`constants.ts`** — Shared constants: `SESSION_TIMEOUT_MS` (15 min), `ABSOLUTE_SESSION_TIMEOUT_MS` (12 hr), `ERROR_CODES` , `HTTP_STATUS` .

How used: The API imports these types to type-check query results and response payloads. The web imports them for component props and API response handling. The collaboration server imports session constants directly (`SESSION_TIMEOUT_MS`). This is the primary mechanism preventing type drift between frontend and backend.

Create a diagram of how the web/, api/, and shared/ packages relate to each other.

```
┌──────────────────────────────────────────────────────────────────┐
│                        pnpm workspace                            │
│                                                                  │
│  ┌──────────────┐    imports    ┌──────────────────────────┐    │
│  │  web/        │◄──────────────►│  shared/  (@ship/shared)  │    │
│  │  React+Vite  │               │  Types, constants, helpers │    │
│  │  :5173       │               └──────────────────────────┘    │
│  └──────┬───────┘                              ▲                 │
│         │ REST + WebSocket                     │ imports         │
│         │ API calls                            │                 │
│  ┌──────▼───────┐               ┌──────────────────────────┐    │
│  │  api/        │◄──────────────►│  shared/  (@ship/shared)  │    │
│  │  Express+WS  │               │  (same package, same types) │    │
│  │  :3000       │               └──────────────────────────┘    │
│  └──────────────┘                                                │
│                                                                  │
│  shared/ is built first (pnpm build:shared) so both api/ and     │
│  web/ can import its compiled output.                            │
└──────────────────────────────────────────────────────────────────┘
```

---

2. Data Model

Find the database schema. Map out the tables and their relationships.

Schema: `api/src/db/schema.sql` + migrations in `api/src/db/migrations/`

Table	Purpose
`workspaces`	Top-level tenants; each has a `sprint_start_date` anchor
`users`	Global identity; supports password auth and PIV (x509) auth
`workspace_memberships`	M:M join — user ↔ workspace with role (admin/member)
`workspace_invites`	Email/PIV invite flow with expiry tokens

`sessions`	Session store: 15-min inactivity + 12-hr absolute timeout
`oauth_state`	Temporary PKCE state for PIV OAuth flows (survives restarts)
`documents`	**The core table — every content type lives here**
`document_associations`	Junction table for program/project/sprint/parent relationships
`document_history`	Audit trail of field changes (especially plan/content changes)
`document_snapshots`	Full snapshots before type conversions (for undo)
`document_links`	Backlinks: which doc links to which
`comments`	Inline threaded comments (linked to TipTap mark `commentId`)
`audit_logs`	Compliance-grade log of all actions
`api_tokens`	Long-lived tokens for CLI/external tool auth
`sprint_iterations`	Per-sprint progress entries (pass/fail/in_progress)
`issue_iterations`	Per-issue progress entries
`files`	File uploads (S3-backed)

Key relationships:

- `documents` self-references via `parent_id` (hierarchy with circular reference guard trigger)
- `document_associations` replaces legacy `program_id` / `project_id` / `sprint_id` columns (dropped by migrations 027, 029)
- `person` documents link to `users` via `properties->>'user_id'` (JSONB), not a FK column

---

Understand the unified document model: how does one table serve docs, issues, projects, and sprints?

All content types share the same physical row in `documents`. The `document_type` PostgreSQL ENUM (`wiki` , `issue` , `program` , `project` , `sprint` , `person` , `weekly_plan` , `weekly_retro` , `standup` , `weekly_review`) acts as a discriminator.

Type-specific data lives in the `properties JSONB` column. For example:

- An issue has `properties = { state, priority, assignee_id, estimate, source }`
- A sprint has `properties = { sprint_number, owner_id, start_date, plan, success_criteria }`
- A person has `properties = { user_id, email, capacity_hours, reports_to }`

All types share: `title` , `content` (TipTap JSON), `yjs_state` (Yjs binary for collaboration), `parent_id` , `position` , `created_by` , `visibility` , `archived_at` , `deleted_at` .

What is the document_type discriminator? How is it used in queries?

`document_type` is a PostgreSQL ENUM. It is used in every list query as a `WHERE document_type = 'issue'` filter. The GIN index on `properties` and a composite index on `(workspace_id, document_type)` make these queries efficient.

In the API routes, each resource type has its own router (`issues.ts` , `documents.ts` , `programs.ts` , etc.) that hard-codes the `document_type` filter. The shared `Editor` component on the frontend receives a `documentType`

prop and renders type-specific property sidebars accordingly.

How does the application handle document relationships (linking, parent-child, project membership)?

- **Parent-child hierarchy:** `parent_id` column on `documents`. A DB trigger (`prevent_circular_parent`) walks the ancestor chain to block circular references. Deleting a parent cascades to all children.
- **Program/project/sprint membership:** `document_associations` junction table with `relationship_type` ENUM (`parent` , `project` , `sprint` , `program`). An issue can belong to one program, one project, and one sprint simultaneously via three separate rows in this table.
- **Backlinks:** `document_links` table — when you link document A in document B's content, a row is added for the backlink feature.
- **Person ↔ User:** `documents.properties->>'user_id'` holds the `users.id` (not a FK column). A partial index on this expression enables efficient lookups.

---

3. Request Flow

Pick one user action and trace it from the React component through the API route to the database query and back.

Example: Creating an issue

```
1. User clicks "New Issue" in web/src/
   └── Component calls POST /api/issues

2. Express receives request at api/src/app.ts
   └── Middleware chain:
       a. helmet() — security headers
       b. cors() — origin check
       c. cookieParser()
       d. express-rate-limit (100 req/min prod)
       e. conditionalCsrf — checks X-CSRF-Token header
       f. authMiddleware — validates session cookie from sessions table

3. Route: api/src/routes/issues.ts → POST /
   └── Inserts row into documents table:
       INSERT INTO documents (workspace_id, document_type, title, properties, ...)
       VALUES ($1, 'issue', 'Untitled', '{"state":"triage","priority":"medium",...}', ...)

4. Response: JSON with new document row

5. WebSocket collaboration server (api/src/collaboration/index.ts)
   └── Client connects to ws://host/collaboration/issue:{id}
   └── Server loads Yjs state from DB, sends sync step 1
   └── Client receives state, renders TipTap editor
```

Identify the middleware chain: what runs before every API request?

From `api/src/app.ts` :

1. `helmet()` — sets security headers (CSP, HSTS, etc.)
2. `cors()` — validates origin against `CORS_ORIGIN` env var
3. Rate limiter (general: 100/min prod, 1000/min dev; login: 5 failed/15min)

4. `express.json()` with 10MB limit
5. `cookieParser()`
6. `conditionalCsrf` — CSRF token validation for session-based requests (skipped for Bearer token auth)
7. `authMiddleware` on protected routes — validates `session_id` cookie against `sessions` table, checks both 15-min inactivity and 12-hr absolute timeouts

---

How does authentication work? What happens to an unauthenticated request?

- **Login:** `POST /api/auth/login` — bcrypt password check → generates 256-bit hex session ID → inserts into `sessions` table → sets `session_id` HttpOnly cookie.
- **Session validation:** `authMiddleware` reads `session_id` cookie, queries `sessions` table, checks both timeout conditions. On valid session, updates `last_activity` and attaches `req.user` and `req.workspaceId`.
- **PIV auth:** Optional FPKI/CAIA OAuth PKCE flow. State stored in `oauth_state` table (survives server restarts). PIV users have `password_hash = null`.
- **API tokens:** Bearer token in `Authorization` header, validated against `api_tokens` table (SHA-256 hashed). Bypasses CSRF check.
- **Unauthenticated request:** `authMiddleware` returns `401 Unauthorized` with `{ error: { code: 'UNAUTHORIZED' } }`. WebSocket upgrade also returns HTTP 401 and destroys the socket.

Phase 2: Deep Dive

4. Real-time Collaboration

How does the WebSocket connection get established?

The HTTP server in `api/src/index.ts` creates an HTTP server and passes it to `setupCollaboration()`. The WS server uses `noServer: true` and intercepts the HTTP `upgrade` event. Two WS endpoints exist:

- `/collaboration/{docType}:{docId}` — document sync (Yjs protocol)
- `/events` — real-time notifications (accountability, presence)

On upgrade: rate limit check → session validation (same cookie check as REST) → document visibility check → `wss.handleUpgrade()`.

How does Yjs sync document state between users?

1. Server maintains an in-memory `Y.Doc` per room (keyed by `{docType}:{docId}`).
2. On connect: server sends sync step 1 (state vector). Client replies with sync step 2 (missing updates). Server sends any missing updates back. Full Yjs CRDT handshake.
3. On edit: `doc.on('update')` fires → server broadcasts binary update to all other connections in the room via `ws.send()`. Each client merges the update into its local Y.Doc via `Y.applyUpdate()`.
4. Awareness (cursors, user presence): separate `Awareness` object per room, broadcasts cursor positions to all room members.
5. Persistence: debounced 2-second timer (`schedulePersist`) saves `Y.encodeStateAsUpdate(doc)` to `documents.yjs_state` as binary, and also converts to TipTap JSON for the `content` column (so REST reads get current content without going through Yjs).

What happens when two users edit the same document at the same time?

Yjs uses CRDTs (Conflict-free Replicated Data Types). Concurrent edits are merged deterministically without conflicts. Each client has a unique `clientID`. Operations are stamped with logical clocks. When two updates arrive out of

order, `Y.applyUpdate()` merges them correctly — no "last write wins" overwriting.

If the server is temporarily unreachable, the client buffers updates locally (IndexedDB cache). On reconnect, it sends the buffered updates. The server checks: if the doc was freshly loaded from JSON (API-created), it sends a `messageClearCache` signal so the client clears its IndexedDB before syncing, preventing stale state merge.

How does the server persist Yjs state?

- Primary path: `yjs_state` column stores `Y.encodeStateAsUpdate(doc)` as `BYTEA`. This is the full binary Yjs document state.
- Fallback: `content` column stores TipTap JSON (converted from Yjs via `yjsToJson()`). This allows REST API reads without a Yjs library.
- On load: server prefers `yjs_state` binary; falls back to converting `content` JSON to Yjs if no binary state exists (for API-created docs).
- On disconnect (last user): pending save is flushed immediately. Doc stays in memory 30 seconds for quick reconnect, then evicted.
- Cache invalidation: REST API updates call `invalidateDocumentCache(docId)`, which closes all WS connections to that doc with code 4101, causing clients to reconnect and reload fresh content.

---

5. TypeScript Patterns

What TypeScript version is the project using?

TypeScript `^5.7.2` (latest 5.x series at time of writing).

What are the tsconfig.json settings? Is strict mode on?

Root `tsconfig.json`:

```
{
  "strict": true,                      // ✅ Full strict mode enabled
  "noUncheckedIndexedAccess": true,    // Array/object index access returns T | undefined
  "noImplicitReturns": true,           // All code paths must return
  "noFallthroughCasesInSwitch": true,  // No implicit switch fallthrough
  "target": "ES2022",
  "module": "NodeNext",
  "moduleResolution": "NodeNext",
  "isolatedModules": true,
  "skipLibCheck": true
}
```

Strict mode is **enabled**. `noUncheckedIndexedAccess` is particularly strict — any array index access (e.g., `arr[0]`) returns `T | undefined`, forcing null checks.

How are types shared between frontend and backend (the shared/ package)?

`shared/` is a pnpm workspace package (`@ship/shared`). Both `api/` and `web/` list it as a dependency in their `package.json`. It is compiled first via `pnpm build:shared` (TypeScript → `dist/`). Both packages import from `@ship/shared` and get the same type definitions.

This is the single source of truth for: `DocumentType`, all `*Properties` interfaces, `Document` base type, `IssueState`, `IssuePriority`, session timeout constants, HTTP status codes, and error codes.

Find examples of: generics, discriminated unions, utility types (Partial, Pick, Omit), and type guards in the codebase.

- **Discriminated unions:** `DocumentType` as a string literal union (`'wiki' | 'issue' | 'program' | ...`) used as the discriminant in `switch` statements throughout the API routes and frontend components.
- **Typed document variants:** `WikiDocument` , `IssueDocument` , `ProjectDocument` etc. all extend `Document` and narrow `document_type` and `properties` to their specific subtypes — a discriminated union pattern.
- **Utility types:** `Partial<ProjectProperties>` used in `DEFAULT_PROJECT_PROPERTIES` . `Pick` and `Omit` used throughout route handlers to subset API response shapes.
- **`ApprovalState` :** `null | 'approved' | 'changed_since_approved' | 'changes_requested'` — a literal union used as a state machine type.
- **`ICEScore` :** `1 | 2 | 3 | 4 | 5` — numeric literal union restricting allowed values.
- **Generics:** API response envelopes use generic wrappers. `computeICEScore()` uses strict typing with `number | null` parameters.

Are there any patterns you do not recognize? Research them.

- **`noUncheckedIndexedAccess` :** A stricter-than-strict TS setting. `arr[0]` returns `T | undefined` even if you know the array is non-empty. Forces defensive coding on all array accesses. Not commonly enabled in most codebases.
- **`csrf-sync` :** Synchronous CSRF token implementation (vs. the older `csurf` package which was deprecated). Uses a double-submit cookie pattern.
- **Yjs awareness protocol:** A separate CRDT for "ephemeral" state (cursor positions, user names) that doesn't need persistence. Has its own `clock` and `update` mechanics distinct from the document CRDT.

---

6. Testing Infrastructure

How are the Playwright tests structured? What fixtures are used?

Tests live in `e2e/` (71 spec files). The fixture system is in `e2e/fixtures/isolated-env.ts` .

Isolation strategy: Each Playwright worker gets its own:

- PostgreSQL container (via `@testcontainers/postgresql`) — spun up fresh per worker
- API server process (on a worker-specific port range, base 10000)
- Vite **preview** server (not dev server — 30-50MB vs 300-500MB per instance)

This eliminates shared-state test flakiness. Worker port ranges: `10000 + (workerIndex * 100)` to avoid collisions.

Fixtures provided: `dbContainer` , `apiServer` , `webServer` , `baseURL` (overridden to point at the isolated web server). A context-level `addInitScript` disables the action items modal globally.

How does the test database get set up and torn down?

Setup (per worker, before tests):

1. `PostgreSqlContainer('postgres:15')` starts a fresh container
2. `runMigrations()` applies `schema.sql` then marks all migration files as applied (skips re-running them since schema.sql already includes their effect)
3. `seedMinimalTestData()` creates: 1 workspace, 2 users, 5 programs, sprints (current±2), 24+ Ship Core issues across all states, 4 projects, wiki documents

Teardown:

- `container.stop()` in a `try/finally` block guarantees cleanup even on errors
- API and web server processes are killed with `SIGTERM` in `try/finally` blocks

---

Run the full test suite. How long does it take? Do all tests pass?

Setup required before tests can run (packages were installed inside Docker containers, not locally):

```
# 1. Install local dependencies (node_modules is currently empty)
pnpm install

# 2. Install Playwright browser binaries
pnpm exec playwright install
```

Then use the `/e2e-test-runner` skill to run tests safely. Do NOT run `pnpm test:e2e` directly — CLAUDE.md warns it causes "output explosion (600+ tests crash Claude Code)."

Metric	Result
Total tests	869
Pass / Fail / Flaky	0 / 869 / 0
Suite runtime	~40 min
Consistently failing tests	All 869 — Windows-only bug (see note below)

Root cause: `e2e/fixtures/isolated-env.ts:231` uses `spawn('npx', ['vite', 'preview', ...])`. On Windows, `npx` is a `.cmd` batch file — Node's `spawn()` without `shell: true` cannot find it, producing `Error: spawn npx ENOENT`. This is a test-infrastructure incompatibility; the suite was authored on macOS/Linux. The application code itself (API, web, shared) builds and runs correctly on Windows. Fix would be adding `shell: process.platform === 'win32'` to that spawn call, or using `node_modules/.bin/vite` directly.

---

7. Build and Deploy

Read the Dockerfile. What does the build process produce?

`Dockerfile.dev` (used by `docker-compose.local.yml`):

- Base: `node:20-slim`
- Installs `pnpm@10`
- Installs only `@ship/api` and `@ship/shared` dependencies (web excluded)
- Builds: `pnpm build:shared && pnpm --filter @ship/api build` → outputs to `api/dist/`
- CMD: `node dist/db/migrate.js && node dist/db/seed.js && node dist/index.js`
- Result: A compiled Node.js Express + WebSocket server at port 3000

`Dockerfile.web` (used by `docker-compose.local.yml`):

- Base: `node:20-slim`
- Installs `@ship/web` and `@ship/shared` dependencies
- Builds `pnpm build:shared` only (web runs as Vite dev server)
- CMD: `pnpm dev` (Vite dev server on port 5173)

- Result: Vite dev server serving the React frontend

Production Dockerfile (`Dockerfile`):

- Runs `pnpm build` to produce a static bundle in `web/dist/`
- Deploys static assets to S3/CloudFront (not a running process)

Read the docker-compose.yml. What services does it start?

File	Services
`docker-compose.yml`	`postgres` only (port 5432) — for native dev where you run API/web locally
`docker-compose.local.yml`	`postgres` (port 5433) + `api` + `web` — full stack in Docker

`docker-compose.local.yml` service dependency: `web` → `api` → `postgres` (with health check). The API won't start until Postgres passes `pg_isready` .

Skim the Terraform configs. What cloud infrastructure does the app expect?

Three environments: `dev` , `shadow` (UAT), `prod` .

Component	AWS Service
API server	AWS Elastic Beanstalk (EC2-backed)
Database	Amazon Aurora (PostgreSQL-compatible) — `terraform/modules/aurora/`
Frontend	S3 + CloudFront — `terraform/modules/cloudfront-s3/`
Secrets	AWS Systems Manager Parameter Store (SSM) — loaded at API startup in production
Security	WAF (`terraform/waf.tf`) + Security Groups
Networking	VPC with public/private subnets — `terraform/modules/vpc/`

Production URL: `https://ship.awsdev.treasury.gov` (CloudFront). API: `ship-api-prod.eba-xsaqsg9h.us-east-1.elasticbeanstalk.com` .

How does the CI/CD pipeline work (if configured)?

No `.github/workflows/` directory exists — **there is no automated CI/CD pipeline** in the repository. Deployments are manual:

- Backend: `./scripts/deploy.sh prod` → builds and deploys to Elastic Beanstalk
- Frontend: `./scripts/deploy-frontend.sh prod` → builds and deploys to S3/CloudFront

Phase 3: Synthesis

8. Architecture Assessment

What are the 3 strongest architectural decisions in this codebase? Why?

1. **Unified document model (everything is a document).** Eliminates a whole class of complexity: no join tables between different entity types, no different migration paths per type, no diverging API patterns.

Adding a new document type (like `weekly_plan`) is a matter of adding an enum value and a properties interface — the storage, collaboration, and linking infrastructure already supports it.

2. **Yjs CRDTs for real-time collaboration with server-side persistence.** The choice to persist both binary Yjs state AND TipTap JSON means the collaboration layer and the REST API are decoupled. REST clients get current content without a Yjs library. The cache invalidation pattern (close WS connections on REST update, clients reconnect and reload) is a clean solution to the dual-write problem.

3. **Shared TypeScript package (`@ship/shared`).** Type definitions are authored once and consumed by both frontend and backend. Session timeout constants are defined in one place and used in both the REST auth middleware and the WebSocket auth handler. This is the correct answer to frontend/backend type drift.

What are the 3 weakest points? Where would you focus improvement?

1. **No CI/CD pipeline.** Manual deployments from developer machines are error-prone and not reproducible. Any developer can push to production. There are no automated tests run before deploy, no artifact versioning.

2. **Type safety violations likely exist despite strict mode.** The `properties JSONB` column stores heterogeneous data but the TypeScript interface at the API layer is `Record<string, unknown>` in the base `Document` type. Individual routes cast to specific property interfaces, but there is no runtime validation (no Zod, no JSON Schema) ensuring DB data matches the TypeScript types. Silent type mismatches are possible.

3. **WebSocket collaboration server is entirely in-memory.** The `docs`, `awareness`, and `conns` Maps are process-local. Horizontal scaling is impossible without a pub/sub layer (Redis). If the API process restarts, all in-flight Yjs updates not yet persisted (within the 2-second debounce window) are lost.

If you had to onboard a new engineer to this codebase, what would you tell them first?

1. **Everything is a document.** Resist the urge to add a new table for a new content type. Read `docs/unified-document-model.md` before writing any SQL.
2. **The `document_associations` table replaced all the relationship columns.** Don't use `program_id`, `project_id`, or `sprint_id` — they were dropped by migrations. All associations go through the junction table.
3. **`pnpm docker:up` is all you need to start.** Ignore the multi-step README; the Docker Compose file handles migrations and seeding automatically.
4. **The 4-panel layout is canonical.** Every editor page has Icon Rail → Sidebar → Content → Properties. Don't break this pattern.

What would break first if this app had 10x more users?

The **WebSocket collaboration server** would break first. The in-memory `docs` and `conns` Maps are process-local. At 10x load, either:

- A single process runs out of memory holding all active Y.Doc objects, or
- Scaling to multiple API instances breaks collaboration (users on different instances can't sync)

The second failure point would be the **PostgreSQL connection pool** in `api/src/db/client.ts`. The default `pg.Pool` is sized for single-instance use. At 10x concurrent API connections the pool would queue and time out.

The **Terraform configs show single-instance Elastic Beanstalk** (no auto-scaling group visible in a quick skim). That single EC2 instance is a hard ceiling.

Discovery Requirement

Find 3 things in this codebase that you did not know before.

Discovery 1: Yjs CRDTs for Real-time Collaboration

What it is: Yjs is a Conflict-free Replicated Data Type (CRDT) library for real-time collaborative editing.

Where found: `api/src/collaboration/index.ts` (entire file), `web/src/` (TipTap editor integration)

What it does and why it matters: Yjs represents a shared document as a CRDT — a data structure that can be updated independently by multiple users and merged automatically without conflicts. Unlike "last write wins" approaches, two users typing simultaneously both have their changes preserved and merged deterministically, regardless of network order. The server holds the authoritative state and broadcasts binary update messages to all clients in a room. This eliminates the need to implement complex operational transformation (OT) logic manually.

How to apply in a future project: Any collaborative editing feature (shared notes, pair programming tools, whiteboards) can use Yjs + a WebSocket relay instead of building custom conflict resolution. The TipTap editor has first-class Yjs support, making rich-text collaboration straightforward to add.

Discovery 2: Shared TypeScript Package in a Monorepo

What it is: A dedicated `shared/` pnpm workspace package (`@ship/shared`) that defines all TypeScript types and constants used by both the frontend and backend.

Where found: `shared/src/types/document.ts` , `shared/src/constants.ts` , imported in both `api/src/` and `web/src/`

What it does and why it matters: In most projects, frontend and backend types drift over time — the API returns a field the frontend doesn't expect, or a constant is defined twice with different values. The shared package solves this structurally. The `SESSION_TIMEOUT_MS` constant, for example, is used in both the REST auth middleware and the WebSocket auth handler from the same source. The `DocumentType` discriminated union is the same type checked in both Express route handlers and React components. This is compile-time enforcement of frontend/backend contracts.

How to apply in a future project: Any TypeScript full-stack project (Next.js, Express+React, etc.) should have a `shared/` package or equivalent for types, enums, and constants the two sides must agree on. pnpm workspaces make this easy to set up with zero build infrastructure overhead.

Discovery 3: Terraform for Cloud Infrastructure as Code

What it is: Terraform is a tool for defining cloud infrastructure (servers, databases, networking, CDN) in `.tf` configuration files that can be version-controlled, reviewed, and applied reproducibly.

Where found: `terraform/` directory — `elastic-beanstalk.tf` , `database.tf` , `s3-cloudfront.tf` , `waf.tf` , `vpc.tf` , modules in `terraform/modules/`

What it does and why it matters: Instead of clicking through the AWS console to create an EC2 instance, RDS database, CloudFront distribution, and WAF rules, everything is declared in code. The `terraform apply` command creates or updates all resources to match the declared state. Three environments (dev, shadow, prod) share the same module structure with different variable values. This means the staging environment is provably configured the same as production — no "works in dev, breaks in prod" due to infrastructure differences.

How to apply in a future project: Any project deploying to AWS (or Azure/GCP) should use Terraform or a similar IaC tool (Pulumi, CDK) rather than manual console configuration. Infrastructure changes get code review, rollback capability, and an audit trail — the same benefits as source code version control.

AI Cost Analysis

Total AI cost for this phase: $0

This project was completed using a subscription-based LLM (Claude Code via Anthropic subscription). Per Gauntlet guidelines, subscription costs are not counted as per-project AI spend.

Tools used:

- Claude Code (Anthropic subscription, prepaid subscription that expires on March 19, 2026) — codebase exploration, checklist completion, documentation generation

Reflection:

- AI was most helpful for: rapid parallel file reading across a large codebase, synthesizing architecture from disparate files, and generating structured documentation from code analysis
- AI was least helpful for: items requiring live execution (test suite results, runtime benchmarks) — those require you to actually run the commands
- Percentage of checklist content AI-generated vs. hand-written: ~90% AI-generated from code analysis, ~10% requires your direct observation and personal reflection (test results, discovery write-up validation)

Appendix A: Installation Process Evaluation

Installation Process Evaluation

Summary

The README.md installation steps are partially incorrect in ordering and do not reflect the recommended Docker-based path for users who already have Docker Desktop installed. This document captures what actually happened during a fresh installation attempt and recommends specific improvements.

What the README Says (Current Steps)

```
1. Clone the repository
2. pnpm install
3. cp api/.env.example api/.env.local
   cp web/.env.example web/.env
4. docker-compose up -d           ← starts database only
5. pnpm db:seed                   ← create sample data
6. pnpm db:migrate                ← run migrations
7. pnpm dev                       ← start the app
```

What Actually Happened (and Why Steps Failed)

Step 3 — Environment files: premature and misleading

Copying `api/.env.example` to `api/.env.local` writes Docker-specific credentials (`ship:ship_dev_password@localhost:5432/ship_dev`) as the DATABASE_URL. This is only correct if the user is using the basic `docker-compose.yml` . If they later follow the `pnpm dev` path (native Postgres) or the `pnpm docker:up` path, this file is wrong or unnecessary. The README gives no guidance on which scenario the user is in.

Step 4 — `docker-compose up -d` failed with a port conflict

The base `docker-compose.yml` maps Postgres to host port `5432` . On any machine where port 5432 is already in use — including Docker Desktop users running other containers — this command fails with:

```
Bind for 0.0.0.0:5432 failed: port is already allocated
```

The README does not warn about this, nor does it mention the alternative `docker-compose.local.yml` file that maps Postgres to port `5433` specifically to avoid this conflict.

Steps 5 & 6 — Seed before migrate is the wrong order

The README lists seeding (step 5) before migrations (step 6). This is backwards: migrations must be applied first to create the schema, then seed data can be inserted. Running `pnpm db:seed` against an empty or partially-migrated schema will fail.

Step 7 — `pnpm dev` is not the right command when using Docker

`pnpm dev` starts the API and web servers as local Node.js processes. It expects a locally accessible Postgres instance and will attempt to create its own `api/.env.local` if one does not exist. If the user followed the Docker path, this is the wrong command. The correct all-in-one Docker command is `pnpm docker:up` .

What actually worked

```
pnpm docker:up
```

This single command (defined in `package.json` as `docker compose -f docker-compose.local.yml up --build`) does everything correctly and in the right order:

1. Pulls and starts Postgres on port **5433** (no conflict with other containers)
2. Builds the API and web Docker images
3. Starts the API container, which **automatically runs migrations then seeds** on first start
4. Starts the web container (Vite dev server)

No separate `pnpm install` , `pnpm db:migrate` , `pnpm db:seed` , or `pnpm dev` steps are needed. The database was fully seeded and ready in one command.

Specific README Improvements Recommended

1. Split into two distinct paths up front

The current README blends the Docker path and the native path without distinguishing them. Users with Docker Desktop should not be running `pnpm dev` ; users without Docker should not be running `docker-compose` . Add a clear fork at the top of the Getting Started section:

```
### Choose your setup path

| Path | When to use |
|------|-------------|
| **Docker (recommended)** | You have Docker Desktop installed |
| **Native** | You have PostgreSQL installed locally |
```

2. For the Docker path, replace steps 3–7 with one command

Remove the multi-step Docker instructions and replace them with:

```
# 3. Start everything (database + API + web)
pnpm docker:up
```

Add a note that this handles migrations and seeding automatically on first run. Remove the separate `pnpm install`, `pnpm db:seed`, `pnpm db:migrate`, and `pnpm dev` steps from the Docker path entirely.

3. Fix the seed/migrate order in the native path

If the native path is kept, fix the step order:

```
# 5. Run database migrations (must come before seeding)
pnpm db:migrate

# 6. Create sample data
pnpm db:seed
```

4. Add a port conflict warning for `docker-compose up -d`

If the basic `docker-compose.yml` is kept in the docs at all, add a warning:

```
> **Note:** If port 5432 is already in use (e.g., another PostgreSQL container),
> use `pnpm docker:up` instead, which runs on port 5433.
```

5. Clarify env file setup per path

- **Docker path:** No need to copy env files manually. `docker-compose.local.yml` injects all environment variables directly into the containers.
- **Native path:** Copy `api/.env.example` to `api/.env.local` and update `DATABASE_URL` to match your local Postgres credentials.

The current instruction to copy both files unconditionally is misleading for Docker users.

6. Surface `pnpm docker:up` prominently

`pnpm docker:up` is the correct entry point for Docker Desktop users but is buried in `package.json` and `docker-compose.local.yml` with no mention in the README. It should appear in both the Getting Started section and the Common Commands table.

Root Cause

The README was likely written describing the native-Postgres developer workflow (the CLAUDE.md instructions confirm local Postgres is the primary dev setup for core contributors). The Docker path was added later via `docker-compose.local.yml` but the README was never updated to reflect it as the recommended approach for new users or those without local Postgres. The result is a README that points users at a command (`docker-compose up -d`) that conflicts with the more complete and conflict-safe alternative (`pnpm docker:up`).

Appendix B: Improved README

Ship logo

Ship

Project management that helps teams learn and improve

License MIT | PRs welcome | Section 508 Compliant | WCAG 2.1 AA

What is Ship?

Ship is a project management tool that combines documentation, issue tracking, and plan-driven weekly workflows in one place. Instead of switching between a wiki, a task tracker, and a spreadsheet, everything lives together.

Built by the U.S. Department of the Treasury for government teams, but useful for any organization that wants to work more effectively.

How to Use Ship

Ship has four main views, each designed for different questions:

View	What it answers
Docs	"Where's that document?" — Wiki-style pages for team knowledge
Issues	"What needs to be done?" — Track tasks, bugs, and features
Projects	"What are we building?" — Group issues into deliverables
Teams	"Who's doing what?" — See workload across people and weeks

The Basics

1. **Create documents** for anything your team needs to remember — meeting notes, specs, onboarding guides
2. **Create issues** for work that needs to get done — assign them to people and track progress
3. **Group issues into projects** to organize related work
4. **Write weekly plans** to declare what you intend to accomplish each week

Everyone on the team can edit documents at the same time. You'll see other people's cursors as they type.

The Ship Philosophy

Everything is a Document

In Ship, there's no difference between a "wiki page" and an "issue" at the data level. They're all documents with different properties. This means:

- You can link any document to any other document
- Issues can have rich content, not just a title and description
- Projects and weeks are documents too — they can contain notes, decisions, and context

Plans Are the Unit of Intent

Ship is plan-driven: each week starts with a written plan declaring what you intend to accomplish and ends with a retro capturing what you learned. Issues are a trailing indicator of what was done, not a leading indicator of what to do.

1. **Plan (Weekly Plan)** — Before the week, write down what you intend to accomplish and why
2. **Execute (The Week)** — Do the work; issues track what was actually done
3. **Reflect (Weekly Retro)** — After the week, write down what actually happened and what you learned

This isn't paperwork for paperwork's sake. Teams that skip retrospectives repeat the same mistakes. Teams that write things down learn and improve.

Learning, Not Compliance

Documentation requirements in Ship are visible but not blocking. You can start a new week without finishing the last retro. But the system makes missing documentation obvious — it shows up as a visual indicator that escalates from yellow to red over time.

The goal isn't to check boxes. It's to capture what your team learned so you can get better.

Getting Started

Choose Your Setup Path

Path	When to use
Docker (recommended)	You have Docker Desktop installed
Native	You have PostgreSQL installed and running locally

Setup: Docker (recommended)

Prerequisites

- Node.js 20 or newer
- pnpm (`npm install -g pnpm`)
- Docker Desktop

Steps

```
# 1. Clone the repository
git clone https://github.com/US-Department-of-the-Treasury/ship.git
```

```
cd ship

# 2. Install dependencies
pnpm install

# 3. Build and start everything (database, API, web)
pnpm docker:up
```

That's it. `pnpm docker:up` handles the rest automatically:

- Starts PostgreSQL on port **5433** (avoids conflicts with any existing PostgreSQL on 5432)
- Builds and starts the API and web containers
- Runs all database migrations
- Seeds the database with sample data

> ***Note:*** *The first run downloads Docker images and installs dependencies inside containers, which takes a few minutes. Subsequent starts are fast.*

Open the App

Once you see `VITE ready` in the output, open your browser to:

http://localhost:5173

Log in with the demo account:

- **Email:** `dev@ship.local`
- **Password:** `admin123`

What's Running

Service	URL	Description
Web app	http://localhost:5173	The Ship interface
API server	http://localhost:3000	Backend services
Swagger UI	http://localhost:3000/api/docs	Interactive API documentation
OpenAPI spec	http://localhost:3000/api/openapi.json	OpenAPI 3.0 specification
PostgreSQL	localhost:5433	Database (Docker, port 5433)

Common Commands (Docker)

```
pnpm docker:up      # Build and start all services
pnpm docker:down    # Stop all services
pnpm docker:clean   # Stop and delete volumes (resets database to fresh state)
```

Setup: Native PostgreSQL

Use this path if you have PostgreSQL installed locally and prefer to run the API and web servers as native Node.js processes.

Prerequisites

- Node.js 20 or newer
- pnpm (`npm install -g pnpm`)
- PostgreSQL 14 or newer, running locally

Steps

```
# 1. Clone the repository
git clone https://github.com/US-Department-of-the-Treasury/ship.git
cd ship

# 2. Install dependencies
pnpm install

# 3. Configure environment
cp api/.env.example api/.env.local
# Edit api/.env.local and set DATABASE_URL to match your local PostgreSQL credentials

cp web/.env.example web/.env

# 4. Run database migrations (must run before seeding)
pnpm db:migrate

# 5. Seed the database with sample data
pnpm db:seed

# 6. Start the application
pnpm dev
```

> ***Note:*** *`pnpm dev` auto-creates `api/.env.local` with a local DATABASE_URL if the file does not exist. If you already copied `.env.example`, verify the `DATABASE_URL` matches your local PostgreSQL setup before running.*

Open the App

http://localhost:5173

Log in with the demo account:

- **Email:** `dev@ship.local`
- **Password:** `admin123`

What's Running

Service	URL	Description
Web app	http://localhost:5173	The Ship interface
API server	http://localhost:3000	Backend services
Swagger UI	http://localhost:3000/api/docs	Interactive API documentation
OpenAPI spec	http://localhost:3000/api/openapi.json	OpenAPI 3.0 specification
PostgreSQL	localhost:5432	Your local PostgreSQL instance

Common Commands (Native)

```
pnpm dev          # Start API and web servers
pnpm dev:web      # Start just the web app
pnpm dev:api      # Start just the API
pnpm db:migrate   # Run database migrations
pnpm db:seed      # Reset database with sample data
pnpm test         # Run tests
```

Technical Details

Architecture

Ship is a monorepo with three packages:

- **web/** — React frontend with TipTap editor for real-time collaboration
- **api/** — Express backend with WebSocket support
- **shared/** — TypeScript types used by both

Tech Stack

Layer	Technology
Frontend	React, Vite, TailwindCSS
Editor	TipTap + Yjs (real-time collaboration)
Backend	Express, Node.js
Database	PostgreSQL
Real-time	WebSocket

Design Decisions

- **Everything is a document** — Single `documents` table with a `document_type` field
- **Server is truth** — Offline-tolerant, syncs when reconnected
- **Boring technology** — Well-understood tools over cutting-edge experiments

See docs/application-architecture.md for more.

Repository Structure

```
ship/
├── api/                    # Express backend
│   ├── src/
│   │   ├── routes/         # REST endpoints
│   │   ├── collaboration/  # WebSocket + Yjs sync
│   │   └── db/             # Database queries
│   └── package.json
│
├── web/                    # React frontend
│   ├── src/
```

```
│   │   ├── components/     # UI components
│   │   ├── pages/          # Route pages
│   │   └── hooks/          # Custom hooks
│   └── package.json
│
├── shared/                 # Shared TypeScript types
├── e2e/                    # Playwright E2E tests
└── docs/                   # Architecture documentation
```

Testing

```pnpm
# Run all E2E tests
pnpm test

# Run tests with UI
pnpm test:ui

# Run specific test file
pnpm test e2e/documents.spec.ts
```

Ship uses Playwright for end-to-end testing covering all major functionality.

Deployment

Ship supports multiple deployment patterns:

Environment	Recommended Approach
Development	Docker Compose (`pnpm docker:up`)
Staging	AWS Elastic Beanstalk
Production	AWS GovCloud with Terraform

Environment Variables

Variable	Description	Default
`DATABASE_URL`	PostgreSQL connection string	Required
`SESSION_SECRET`	Cookie signing secret	Required
`PORT`	API server port	`3000`

Security

- **No external telemetry** — No Sentry, PostHog, or third-party analytics
- **No external CDN** — All assets served from your infrastructure
- **Session timeout** — 15-minute idle timeout (government standard)
- **Audit logging** — Track all document operations

> ***Reporting Vulnerabilities:*** *See SECURITY.md for our vulnerability disclosure policy.*

Accessibility

Ship is Section 508 compliant and meets WCAG 2.1 AA standards:

- All color contrasts meet 4.5:1 minimum
- Full keyboard navigation
- Screen reader support
- Visible focus indicators

Contributing

We welcome contributions. See CONTRIBUTING.md for guidelines.

Documentation

- Application Architecture — Tech stack and design decisions
- Unified Document Model — Data model and sync architecture
- Document Model Conventions — Terminology and patterns
- Week Documentation Philosophy — Why weekly plans and retros work the way they do
- Accountability Philosophy — How Ship enforces accountability
- Accountability Manager Guide — Using approval workflows
- Contributing Guidelines — How to contribute
- Security Policy — Vulnerability reporting

License

MIT License